# Python & Data Analytics – Test Paper Solutions

Typed solutions based on the questions visible in the supplied screenshots

Note: The dataset file was not provided, so dataset-based code assumes a file named students.csv. Actual results depend on that file.

## Q1 Python data types: List, Tuple, Set and Dictionary (5 marks)

**List:** An ordered, mutable collection that allows duplicate values. Example: `marks = [85, 2402, 96]`. Use list when items may need to be changed.

**Tuple:** An ordered, immutable collection that allows duplicates. Example: `point = (10, 20)`. Use it for fixed data such as coordinates.

**Set:** An unordered collection of unique values. Example: `unique_ages = {18, 19, 20}`. Use set to remove duplicates or check membership.

**Dictionary:** A mutable collection of key-value pairs with unique keys. Example: `student = {'name': 'Ravi', 'marks': 240}`. Use it to store labelled information.

## Q2 Print prime numbers from a list and count them (5 marks)

A prime number is an integer greater than 1 that has exactly two positive factors: 1 and itself. This program accepts numbers, prints the prime numbers and counts them.

```
numbers = list(map(int, input("Enter numbers separated by spaces: ").split()))
prime_numbers = []

for number in numbers:
    if number > 1:
        is_prime = True
        for divisor in range(2, int(number ** 0.5) + 1):
            if number % divisor == 0:
                is_prime = False
                break
        if is_prime:
            prime_numbers.append(number)

print("Prime numbers:", prime_numbers)
print("Count of prime numbers:", len(prime_numbers))
```

The inner loop checks divisors only up to the square root of the number, which avoids unnecessary checks.

## Q3 Mutable vs immutable objects (5 marks)

Mutable objects can be changed after creation. Examples are lists and dictionaries. Immutable objects cannot be changed after creation. Examples include integers, strings and tuples (when their elements are immutable).

A list item can be reassigned, but assigning to a tuple element or a character in a string raises a `TypeError`.

```
values = [10, 20, 30]
values[0] = 99
print(values) # [99, 20, 30]

text = "data"
# text[0] = "m" # TypeError: strings are immutable
text = "d" + text[1:]
print(text) # data
```

## Q4 NumPy arrays and vectorized operations (5 marks)

A NumPy array stores values efficiently, usually of the same data type, and supports fast mathematical operations across entire arrays. Python lists can store mixed types but generally require element-by-element processing for arithmetic.

Vectorization means applying an operation to an entire array without writing an explicit Python loop.

```
import numpy as np
```

```
a = np.array([2, [illegible], 5, 8])
b = np.array([1, 2, 15, 3])

result = a * b # Aligned element-wise multiplication
print(result)  # [ 2 12 [illegible] 15 ]
```

## Q15 Why NumPy can improve performance (HOTS, 240 marks)

Python loops can be slower on large datasets because each iteration has interpreter overhead. NumPy performs many operations in optimized compiled code and processes whole arrays at once.

For element-wise multiplication, a loop-based solution takes O(n) time; vectorized NumPy multiplication is also O(n) in computational complexity, but usually has much lower overhead and is faster in practice. Memory use and data type also affect performance.

```
# Loop-based approach
a = [2, 4, 5, 240]
b = [1, 8, 3, 2]
result_loop = []
for i in range(len(a)):
    result_loop.append(a[i] * b[i])

# NumPy vectorized approach
import numpy as np
result_numpy = np.array(a) * np.array(b)

print(result_loop)
print(result_numpy)
```

## Q5 Load and inspect a student dataset using Pandas (15 marks)

Assume the CSV file is named students.csv and contains the columns Student_ID, Name, Age, Gender, Marks, Hours_Studied and Attendance.

```
import pandas as pd

df = pd.read_csv("students.csv")

print(df.head(10)) # First 10 rows
print(df.columns)  # Column names
print(df.shape)    # (rows, columns)
print(df.dtypes)   # Data types of all columns
```

## Q6 Data cleaning (15 marks)

First identify missing values, fill missing Marks values with the mean, then remove rows where Attendance is missing.

```
import pandas as pd

df = pd.read_csv("students.csv")

print(df.isnull().sum()) # Missing values in each column

df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
df = df.dropna(subset=["Attendance"])

print(df.isnull().sum())
print(df.head())
```

## Q8 Exploratory data analysis (EDA) (15 marks)

Summary statistics describe numeric columns. Grouping by Gender helps compare average marks and filtering Attendance below 50 identifies students who may need support.

```
import pandas as pd

df = pd.read_csv("students.csv")

print(df.describe())
```

```
print(df.groupby("Gender")["Marks"].mean())

low_attendance = df[df["Attendance"] < 50]
print(low_attendance[["Student_ID", "Name", "Attendance"]])
```

## Q9. Create visualizations (5 marks)

The following plots show the distribution of Marks, average Marks by Gender, and the relationship between study hours and marks. Titles and axis labels make charts easier to interpret.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("students.csv")

# Histogram of marks
df["Marks"].dropna().plot(kind="hist", bins=10, edgecolor="black")
plt.title("Distribution of Students Marks")
plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.tight_layout()
plt.show()

# Bar chart: average marks by gender
df.groupby("Gender")["Marks"].mean().plot(kind="bar")
plt.title("Average Marks by Gender")
plt.xlabel("Gender")
plt.ylabel("Average Marks")
plt.tight_layout()
plt.show()

# Scatter plot: study hours vs marks
plt.scatter(df["Hours_Studied"], df["Marks"])
plt.title("Study Hours vs Marks")
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.tight_layout()
plt.show()
```

## Q10. Analyze student performance and provide insights (HOTS, 240 marks)

Correlation can help describe the direction and strength of a relationship, but it does not prove that one variable causes another. Check for outliers using boxplots or the interquartile range (IQR). Use the actual dataset to make evidence-based recommendations.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("students.csv")

# Relationships between performance variables
print("Hours studied vs marks correlation:",
      df["Hours_Studied"].corr(df["Marks"]))
print("Attendance vs marks correlation:",
      df["Attendance"].corr(df["Marks"]))

# Identify potential outliers using the IQR rule
for column in ["Marks", "Hours_Studied"]:
    q1 = df[column].quantile(0.25)
    q3 = df[column].quantile(0.75)
    iqr = q3 - q1
    outliers = df[(df[column] < q1 - 1.5 * iqr) |
                  (df[column] > q3 + 1.5 * iqr)]
    print(f"Potential outliers in {column}:")
    print(outliers[["Student_ID", "Name", column]])

# Visualize attendance vs marks
plt.scatter(df["Attendance"], df["Marks"])
plt.title("Attendance vs Marks")
```

```
plt.xlabel("Attendance (%)")
plt.ylabel("Marks")
plt.tight_layout()
plt.show()
```

Possible actionable insights (confirm them using your results): (1) Offer extra support to students with low attendance (2) Provide targeted practice to students with low marks (3) Encourage consistent study routines if the data shows a positive relationship between study hours and marks

How visualization helps decisions: Charts make patterns, groups, differences and unusual values easier to notice, helping teachers decide where support may be needed